Mestrado em Engenharia Informática
Armazéns de Dados · 1.º Semestre 2026-2027

Data Mart de Vendas da More Rhythm, SA

Trabalho Prático — 1.ª parte: arquitetura, modelo dimensional, Staging Area e mapeamentos

Grupo: alunos n.º 1230881 · 1230977

Entrega: 15 de novembro de 2026

Índice

1 Introdução
2 Arquitetura do armazém de dados
3 Data Profiling
4 Modelo dimensional
5 Staging Area
6 Mapeamentos
7 Regras de transformação, limpeza e rejeição
8 Carga incremental
9 Uniformização monetária em euros
10 Decisões em aberto e limitações
Anexo A — Matriz de requisitos analíticos
Anexo B — Dicionário de dados e estratégias SCD

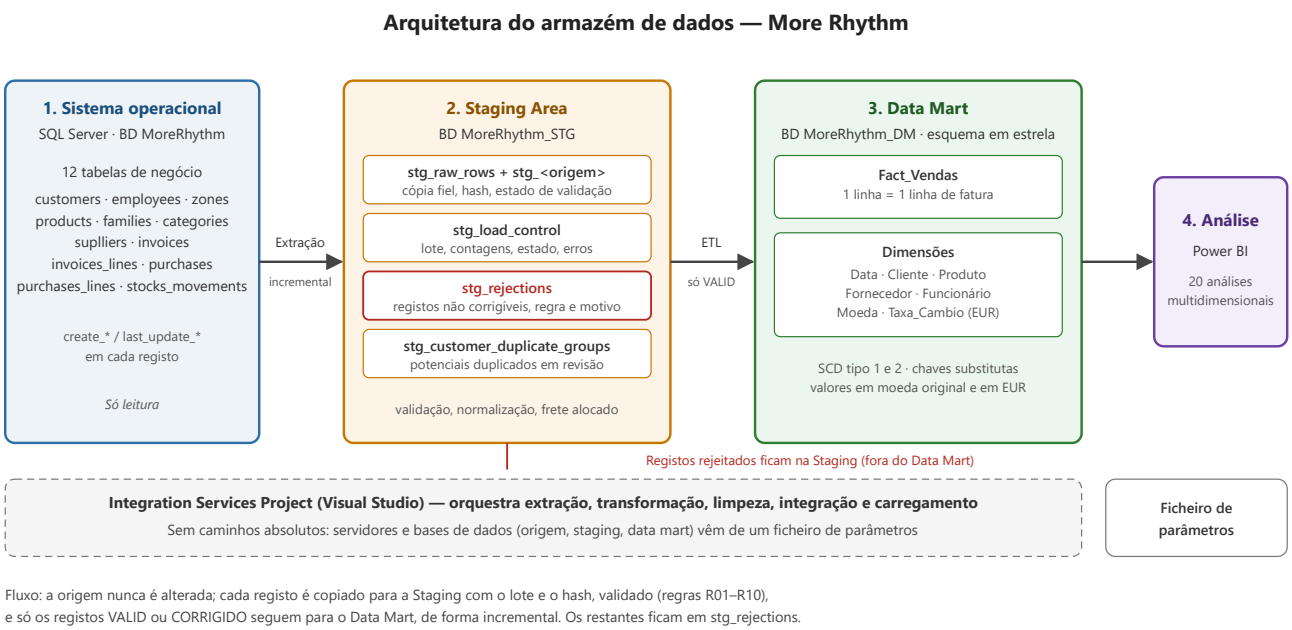
1 Introdução

Este relatório documenta a 1.ª parte do trabalho prático de Armazéns de Dados: o desenho de um *data mart* de vendas para a More Rhythm, SA, empresa sediada no Porto que distribui instrumentos musicais em Portugal e, pontualmente, em Espanha. A gestão quer analisar historicamente as vendas (faturação) a clientes, ao nível de detalhe mais elementar.

O relatório apresenta a arquitetura do armazém de dados, o resultado do *Data Profiling* à base de dados operacional, o modelo dimensional (metodologia de Kimball) com os respetivos atributos e tipos de dados, as estruturas da *Staging Area*, os mapeamentos origem → Staging e Staging → Data Mart (com a estratégia de *Slowly Changing Dimension* de cada atributo) e as regras de transformação e limpeza.

Os dados disponibilizados são uma amostra da base `MoreRhythm`: 12 tabelas de negócio e 1 057 registos, dos quais 37 faturas e 93 linhas de fatura, com datas entre 31/01/2025 e 01/05/2025. Algumas análises do enunciado (por exemplo a da primavera e a do 2.º trimestre) devolvem por isso poucos dados; isso não é um erro do modelo.

2 Arquitetura do armazém de dados



Arquitetura do armazém de dados

A arquitetura tem quatro camadas, ligadas por um processo ETL implementado num *Integration Services Project* do Visual Studio.

Camada	Base de dados	Função
Sistema operacional	`MoreRhythm`	Origem dos dados. É apenas lida; o processo nunca a altera
Staging Area	`MoreRhythm_STG`	Guarda uma cópia fiel e rastreável dos registos extraídos, valida-os, normaliza-os e isola os que têm problemas de qualidade
Data Mart	`MoreRhythm_DM`	Esquema em estrela com uma tabela de factos e as dimensões, pronto para consulta
Análise	Power BI	As 20 análises multidimensionais pedidas

Fluxo.

1. **Extração:** cada tabela de origem é copiada para a Staging, com o identificador do lote (`load_id`), a data da extração e um *hash* do registo. A extração é incremental sempre que a tabela tem colunas de auditoria (ver secção "Carga incremental").
2. **Validação e limpeza:** aplicam-se as regras R01–R10. Cada registo fica `VALID`, `CORRIGIDO`, `PENDING` ou `REJECTED`. A normalização é feita em cópias de comparação; o valor original nunca se perde.
3. **Rejeição:** os registos que não podem ser corrigidos automaticamente ficam em `stg_rejections`, com a regra violada e o motivo, e **não** são carregados no Data Mart, como o enunciado exige.
4. **Carregamento:** as dimensões carregam antes do facto. Só entram registos `VALID` ou `CORRIGIDO`. O carregamento é incremental e idempotente.
5. **Frete e moeda:** o frete (que na origem está ao nível da fatura) é repartido pelas linhas e os valores são convertidos para euros durante a transformação.

Parametrização. O projeto SSIS não tem caminhos absolutos. O nome do servidor e da base de dados do sistema operacional, o servidor do Data Mart e da Staging (o mesmo para ambas) e os nomes das bases de

dados do Data Mart e da Staging vêm de um ficheiro de parâmetros.

3 Data Profiling

O *profiling* foi feito com seis *scripts* SQL sobre a `MoreRhythm` (inventário, chaves, valores em falta, duplicados de clientes, inconsistências financeiras e integridade referencial). A origem declara 16 chaves estrangeiras e os testes de integridade referencial não encontraram registos órfãos. Os problemas de completude e de consistência estão resumidos abaixo.

Problema	**Evidência**	**Tratamento**
NIF de cliente em falta	1 cliente (e 1 com `000000000`)	`N/D`; fora da deteção de duplicados
Zona de cliente em falta	2 clientes	Não se infere: "Zona desconhecida"
Email e telemóvel em falta	12 e 21 clientes	Mantêm-se `NULL`
Atributos de produto em falta	2 designações, famílias e categorias; 12 fornecedores	Membro "Desconhecido" (descritivos)
Linhas de compra sem produto	12 linhas, quantidade 319, valor 12 615,30	Rejeitadas na Staging; fora do âmbito do data mart de vendas
Clientes potencialmente duplicados	Clientes 10 e 18 (mesmo NIF e nome normalizados)	Mantidos separados, em revisão
Linhas de fatura com `line_total` ≠ quantidade × preço	4 linhas (2 com quantidade negativa, 2 com preço negativo)	Fatura rejeitada até confirmar a regra de sinal
Cabeçalho ≠ soma das linhas	2 faturas, diferenças de +5,00 e −5,00	Fatura rejeitada

Confirmado e por confirmar. Os números acima foram confirmados por *queries* sobre a origem. A interpretação dos casos de sinal negativo (devoluções ou erro de dados) e o significado de `invoices.payment_date` (sempre preenchida, entre 03/02 e 31/12/2025 e nunca anterior à data da fatura, pelo que pode ser uma data de vencimento) dependem de confirmação do negócio e estão assinalados como decisões em aberto.

4 Modelo dimensional

4.1 Metodologia

O data mart foi desenhado segundo a metodologia de Kimball, que organiza a modelação em quatro passos: escolha do processo de negócio, definição da granularidade, identificação das dimensões e identificação dos factos. A gestão da More Rhythm quer analisar historicamente as **vendas a clientes**, por isso o processo de negócio é a **faturação**. O resultado é um esquema em estrela com uma tabela de factos e seis dimensões, que respeita o enunciado (consultas ao nível mais elementar, histórico, Euro como moeda de uniformização).

4.2 Requisitos analíticos

As dez análises do enunciado foram decompostas em medidas, filtros, atributos de agrupamento e operações de *drill down* e *roll up* (matriz no Anexo A). Destas resultaram os requisitos do modelo:

- calendário completo (semestre, trimestre, quadrimestre, mês, semana ISO, estação do ano) e indicadores de fim de semana e último dia do mês;
- duas hierarquias de produto (família e categoria) e geografia do cliente e do fornecedor;
- tipo de cliente como nível de *roll up* distinto da geografia;
- vendas com e sem IVA e custos de expedição ao nível de família e de produto.

As análises são **indicativas**, por isso o modelo inclui também outros atributos (segmento, grupo e chefia do funcionário, nacionalidade do fornecedor, prazo de pagamento) que permitem muitas outras consultas.

4.3 Granularidade

A tabela de factos tem a granularidade de **uma linha de fatura** (um produto numa fatura). Esta escolha justifica-se porque:

1. é o nível mais baixo de detalhe disponível (`invoices_lines`) e o enunciado exige o nível mais elementar;
2. permite cruzar produto, quantidade e valores. Uma tabela ao nível da fatura perderia o produto e impossibilitaria as análises por família, categoria e produto;
3. qualquer agregação superior se obtém por soma, mas o inverso é impossível.

4.3.1 Custos de expedição

O custo de expedição existe apenas no cabeçalho da fatura. Para o tornar aditivo ao nível da linha, é **alocado proporcionalmente ao valor líquido de cada linha**, sendo o resto do arredondamento absorvido pela última linha. Assim, a soma do frete alocado das linhas é igual ao frete da fatura. Foram rejeitadas as alternativas de repetir o valor em todas as linhas (duplicaria ao somar) e de manter uma tabela de factos separada ao nível da fatura (não permitiria a análise 2, por família). A alocação é realizada na Staging Area e validada no ETL.

4.3.2 Moeda

Os valores são guardados na moeda original e em euros, juntamente com a taxa de câmbio do dia da fatura, mantida numa tabela de taxas. A amostra só tem euros, mas a estrutura já suporta a futura sucursal no Reino Unido sem alterações.

4.4 Dimensões e factos

Tabela	Tipo	Conteúdo principal
`Fact_Vendas`	Factos	Quantidade, valor líquido, IVA, frete alocado (moeda original e EUR), fatura (dimensão degenerada)

Tabela	Tipo	Conteúdo principal
`Dim_Data`	Dimensão (2 papéis)	Data da fatura e data de pagamento, com todos os níveis do calendário
`Dim_Cliente`	Dimensão	Dados do cliente, cidade, zona, país, tipo, segmento, vendedor
`Dim_Produto`	Dimensão	Produto, família, categoria, classificação, taxa de IVA
`Dim_Fornecedor`	Dimensão	Fornecedor, cidade, zona, tipo, nacionalidade
`Dim_Funcionario`	Dimensão	Funcionário, grupo, departamento, chefe
`Dim_Moeda`, `Dim_Taxa_Cambio`	Dimensão de apoio	Moedas e taxas de conversão para EUR

O diagrama está na Figura 1 (`diagrama_modelo_dimensional.png`) e o dicionário de dados completo (tipos, chaves, nulabilidade) no Anexo B.

As **medidas** são aditivas (quantidade, valor líquido, IVA, frete alocado); o preço unitário é não aditivo e só se agrega por média. "Vendas com IVA" é `Valor_Liquido + Valor_IVA`, sem frete; "Total faturado" é `Valor_Liquido + Valor_IVA + Frete_Alocado`. Estas medidas são calculadas, para evitar redundância e inconsistências.

Dimensões conformes e chaves: todas as dimensões usam chaves substitutas, mais a chave de negócio para mapeamento. Cada dimensão tem um membro "Desconhecido" (SK −1) para valores em falta que não impeçam o carregamento.

Fornecedor: associa-se à linha de fatura através do produto vendido (`products.supplyer`), o que viabiliza as análises 3 e 10.

4.5 Hierarquias

Dimensão	Hierarquias
Data	Ano > Semestre > Trimestre > Mês > Dia · Ano > Quadrimestre > Mês · Ano > Semana > Dia · Ano > Estação > Dia
Produto	Classificação > Família > Produto · Categoria > Produto
Cliente	País > Zona > Cidade > Cliente · Tipo > Cliente · Segmento > Cliente
Fornecedor	Zona > Cidade > Fornecedor
Funcionário	Departamento > Grupo > Funcionário · Chefe > Funcionário

As hierarquias de produto e de cliente são paralelas e não aninhadas, porque família e categoria são atributos independentes do produto, tal como tipo e zona são do cliente. Os níveis estão **desnormalizados** nas dimensões (estrela, sem *snowflake*), o que simplifica as consultas e melhora o desempenho.

4.6 Estratégias de Slowly Changing Dimensions

O sistema operacional regista data e hora de criação e de cada alteração, mas não conserva os valores antigos; o histórico no data mart começa, portanto, no primeiro carregamento. As alterações são detetadas comparando o registo da origem com a versão atual da dimensão.

- **SCD tipo 1** (substituição) para atributos que apenas se corrigem ou não afetam a análise: nomes, NIF, contactos, moradas, prazos de pagamento, designação do produto.
- **SCD tipo 2** (nova versão, com `Data_Inicio`, `Data_Fim` e `Versao_Atual`) para atributos que classificam as vendas: zona, cidade, tipo e segmento do cliente e vendedor atribuído; família, categoria, classificação e taxa de IVA do produto; zona, cidade e tipo do fornecedor; grupo, departamento e chefe do funcionário.

Justificação: se um cliente mudar de zona, as vendas passadas devem manter-se na zona em que foram realizadas; com tipo 1 as análises históricas seriam alteradas retroativamente. A tabela de factos referencia a versão da dimensão vigente na **data da fatura**. `Dim_Data` e as taxas de câmbio não têm SCD, por serem imutáveis.

4.7 Clientes duplicados

Os clientes potencialmente duplicados não são consolidados automaticamente. Os clientes 10 e 18 permanecem distintos, com o mesmo grupo de revisão na Staging, até existir confirmação explícita; as duas faturas conservam o identificador do respetivo cliente. Na amostra, o data profiling identificou como potenciais duplicados o cliente *Eira e Beira, lda* (NIF 502323434), registado com os números 10 e 18.

4.8 Dados em falta e registos rejeitados

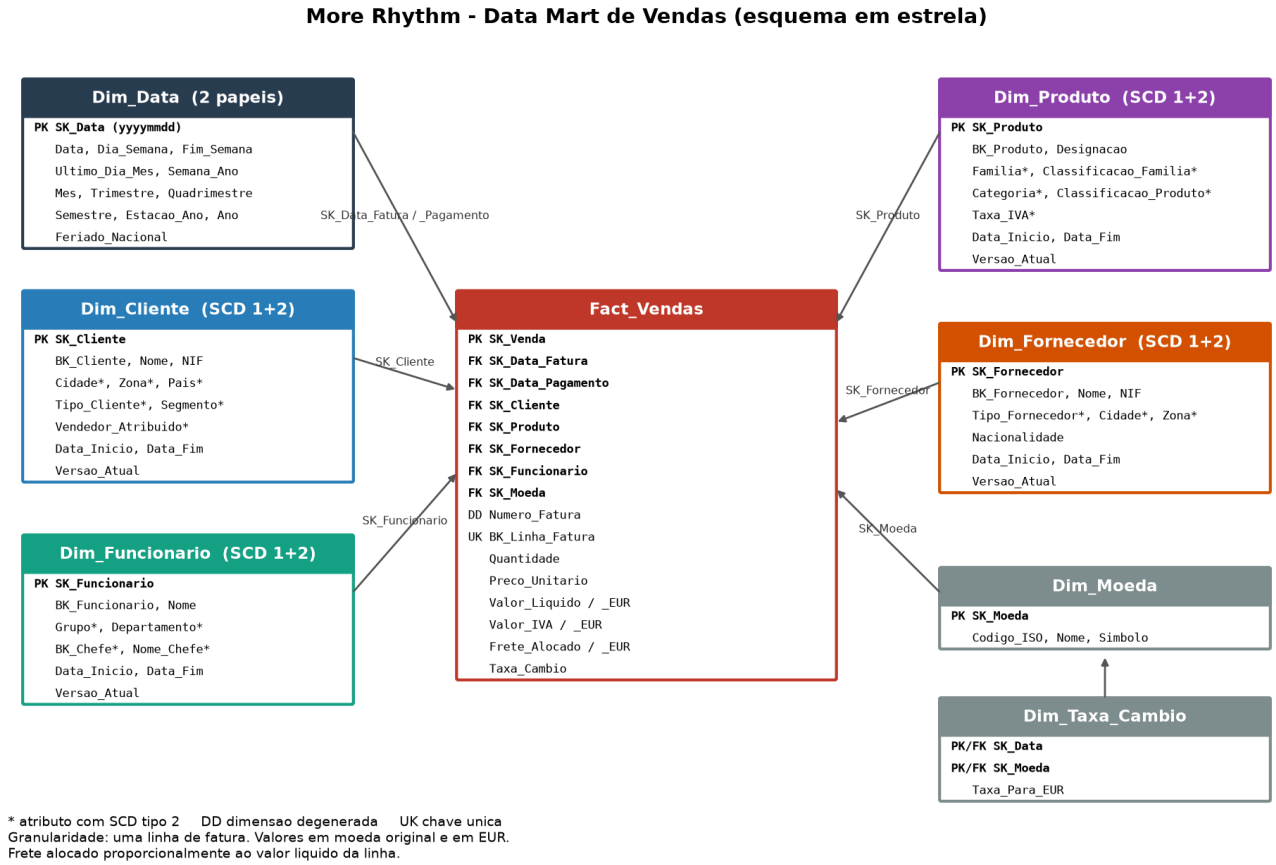
O modelo segue um princípio comum com as regras de limpeza da Staging: um **atributo descritivo em falta** é carregado com um membro "Desconhecido" (por exemplo, produtos sem fornecedor usam `SK_Fornecedor = -1`; clientes sem zona ficam em "Zona desconhecida"), para que a venda continue visível; um problema que afete a **identificação do registo ou as medidas** (linhas de fatura com total incoerente com quantidade × preço, ou faturas cujo cabeçalho diverge da soma das linhas) leva à **rejeição da fatura** para tabelas próprias da Staging. Rejeita-se a fatura inteira, e não só a linha, para que o frete alocado continue a reconciliar com o frete da fatura.

4.9 Suporte a todas as análises

Os atributos pedidos pelas dez análises (semestre, quadrimestre, semana, estação, fim de semana, último dia do mês, zona, cidade, tipo de cliente, categoria, família, produto, fornecedor, zona do fornecedor, funcionário) existem todos no modelo, ao nível da granularidade escolhida. Os níveis de *drill down* e *roll up* exigidos correspondem às hierarquias da secção 5. Um conjunto adicional de análises (evolução temporal, rankings, devoluções, prazo de pagamento, peso do frete, vendas por país) também é suportado (Anexo A, §3).

4.10 Decisões fechadas

- "Vendas com IVA" é `Valor_Liquido + Valor_IVA`, sem frete.
- As 4 linhas com sinal negativo e as 2 faturas com totais divergentes são rejeitadas até validação de negócio.
- "Desconhecido" é reservado a atributos descritivos; chaves, medidas e relações inválidas são rejeitadas.
- Duplicados aproximados ficam em revisão e não são fundidos automaticamente.
- O mapeamento Staging → Data Mart e as regras de rejeição são os definidos em `mappings/source-to-staging.md` e `mappings/transformacoes-limpeza.md`.



Esquema em estrela do Data Mart

A figura mostra a tabela de factos e as dimensões (atributos marcados com `*` têm SCD tipo 2). O dicionário completo de dados, com tipos, chaves e nulabilidade, está no Anexo B. As restrições físicas adicionais (FK composta do facto para as taxas de câmbio, vigência SCD2 e índices únicos de versão corrente) estão descritas na secção seguinte.

4.11 Implementação física

O esquema foi implementado em SQL Server (`04_ddl_data_mart.sql`) e validado com 33 testes automáticos (chaves, restrições, tipos, calendário e SCD2), todos aprovados numa base de dados descartável. A `Dim_Data` é populada de 2020 a 2035, com feriados nacionais (incluindo os móveis, calculados a partir da Páscoa) e as taxas de câmbio EUR = 1. Os tipos das chaves de negócio coincidem com os da origem (`numeric(10,0)` para clientes e fornecedores, `numeric(6,0)` para funcionários).

5 Staging Area

A Staging é a camada onde os dados são preservados, rastreados e validados antes de chegarem ao Data Mart. A estrutura mínima executável (`03_ddl_staging_area.sql`) contém:

Tabela	Função
`stg_load_control`	Um registo por execução: origem, início e fim, contagens (extraídas, validadas, rejeitadas), estado (`RUNNING`, `SUCCEEDED`, `FAILED`) e mensagem de erro
`stg_raw_rows`	Cópia fiel de cada registo (`raw_row_json`) com tabela e chave de origem, instante da extração, *hash* e estado de validação
`stg_rejections`	Um registo por violação de regra, com os valores originais, a regra (R01–R10), a mensagem e o estado de resolução (`OPEN`, `RESOLVED`, `CANCELLED`)
`stg_customer_duplicate_groups`	Potenciais duplicados de clientes, com NIF e nome normalizados e estado de revisão (`PENDING_REVIEW`, `CONFIRMED`, `DISMISSED`)
`stg_<origem>` (12 tabelas)	Cópia tipada de cada tabela de origem, com os mesmos nomes e tipos, mais as colunas técnicas abaixo

5.1 Contrato físico da Staging Area

Cada tabela `stg_<origem>` mantém as colunas de negócio da origem, sem conversão destrutiva, e acrescenta os metadados seguintes:

Coluna	Tipo SQL Server	Regra
`stg_row_id`	`bigint IDENTITY`	Identificador técnico da cópia
`load_id`	`bigint`	Identifica a execução em `stg_load_control`
`source_table`	`sysname`	Nome da tabela de origem
`source_pk`	`nvarchar(250)`	Chave primária serializada de forma estável
`extraction_timestamp`	`datetime2(3)`	Instante UTC da extração
`validation_status`	`varchar(20)`	`VALID`, `CORRIGIDO`, `PENDING` ou `REJECTED`
`validation_rule`	`varchar(10)`	Regra R01–R10, quando aplicável
`validation_message`	`nvarchar(1000)`	Mensagem legível da validação
`raw_row_hash`	`varbinary(32)`	Hash dos valores de origem, para idempotência

O `source_pk` é composto por `coluna=valor` em ordem alfabética das colunas da chave, usando `|` como separador. A chave não depende da ordem física da tabela. A tabela de origem nunca é atualizada pelo processo.

5.1.1 Controlo de lotes e rejeições

`stg_load_control` regista `load_id`, origem, início/fim, contagens extraídas, válidas, rejeitadas e estado do lote. `stg_rejections` regista uma linha por registo e regra violada, incluindo `load_id`, `source_table`, `source_pk`, `raw_row_json`, `validation_rule`, `validation_message`, `rejected_at` e `resolution_status`. Assim, uma rejeição conserva os valores originais mesmo que a estrutura normalizada seja alterada.

As 12 linhas de compras sem produto usam `R05`; as quatro linhas financeiras usam `R06`; as duas faturas divergentes usam `R07`. Uma linha rejeitada por R06 rejeita a fatura inteira, para que o frete só seja alocado sobre um conjunto completo de linhas.

6 Mapeamentos

6.1 Origem → Staging

A cópia é 1:1: cada tabela de origem tem a sua `stg_<origem>`, com as mesmas colunas, tipos, precisão e nulabilidade; os `NULL` mantêm-se `NULL` e nenhuma conversão é destrutiva. A tabela técnica `sysdiagrams` não é copiada. As regras de validação aplicadas na passagem para a Staging são as seguintes.

Origem → Staging	Tipo preservado	Transformação de carga	Validação
Todas as colunas de `categories`, `customers`, `employees`, `families`, `invoices`, `invoices_lines`, `products`, `purchases`, `purchases_lines`, `stocks_movements`, `suplliers`, `zones`	Tipo, precisão e escala da origem	Cópia 1:1; `NULL` permanece `NULL`; espaços só são removidos numa coluna de comparação	Chave primária não nula; hash e metadados obrigatórios
`customers.taxpayer_number`, nome, email e morada	Tipo da origem + colunas normalizadas	`normalised_*` apenas para comparação; vazio torna-se `NULL` na coluna transformada	R01 para colisões; não fundir sem aprovação
`invoices_lines.quantity`, `unit_price`, `line_total`	`decimal(14,4)` / `decimal(19,6)`	Preservar sinal e precisão	R06 quando `line_total <> quantity * unit_price`
`invoices.total_products` e linhas da fatura	`decimal(19,6)`	Calcular `line_sum` e `header_line_difference`	R07 quando `ABS(difference) > 0,01`
`purchases_lines.product`	Tipo da origem	Nenhuma substituição	R05 quando nulo ou vazio

6.2 Staging → Data Mart

O mapeamento seguinte indica, para cada atributo do Data Mart, a origem, as transformações e a estratégia SCD.

O enunciado exige, para cada atributo, a origem, as operações de transformação e limpeza e a estratégia SCD. Os tipos de origem foram confirmados na `MoreRhythm` (`INFORMATION_SCHEMA.COLUMNS`).

6.2.1 Convenções

Convenção	Regra
`limpa(x)`	`LTRIM(RTRIM(x))`; se o resultado for vazio → `NULL`. O valor bruto fica sempre em `raw_row_json`
Auditoria	`Data_Criacao_Origem = create_date + create_time`; `Data_Atualizacao_Origem = last_update_date + last_update_time` (`datetime2(0)`). Se forem nulos na origem (2 produtos na amostra) usa-se `1900-01-01`
`Hash_SCD1` / `Hash_SCD2`	`HASHBYTES('SHA2_256', ...)` dos atributos SCD1 / SCD2 **já transformados**, concatenados por `|` com `NULL` → texto vazio e com `CONCAT`
Só carregam	registos com `validation_status` `VALID` ou `CORRIGIDO` (a Staging usa `PENDING` / `REJECTED`; nos documentos de regras aparecem como "pendente" / "rejeitado")
Vigência SCD2	1.ª versão: `Data_Inicio = 1900-01-01`. Nova versão: `Data_Inicio = max(CAST(last_update_date AS date), Data_Inicio_anterior + 1)`; a anterior fecha com `Data_Fim = Data_Inicio_nova − 1`, `Versao_Atual = 0`

6.2.2 `Dim_Cliente` ← `stg_customers`, `stg_zones`, `stg_employees`

Atributo destino (tipo)	Origem	Transformação / limpeza	SCD
BK_Cliente `numeric(10,0)`	`customers.number`	Cópia	—
Nome `nvarchar(55)`	`customers.name`	`limpa`; vazio → rejeitar (identificação)	1
NIF `varchar(20)`	`customers.taxpayer_number`	Remover espaços e separadores; vazio ou só zeros → `N/D` (R02). Não entra na deteção de duplicados se `N/D`	1
Contacto `nvarchar(30)`	`customers.contact`	`limpa`	1
Email `varchar(45)`	`customers.email`	`limpa`; `NULL` se vazio	1
Telefone `varchar(60)`	`customers.phone`	`limpa`	1
Telemovel `varchar(45)`	`customers.mobile`	`limpa`	1
Morada `nvarchar(55)`	`customers.address`	`limpa`	1
Codigo_Postal `varchar(45)`	`customers.zip_code`	`limpa`	1
Cidade `nvarchar(43)`	`customers.clity` (grafia da origem)	`limpa`; vazio → `Desconhecida`	**2**
Zona `nvarchar(50)`	`zones.zone` via `customers.zone`	`NULL` ou sem correspondência → `Zona desconhecida`; nunca se infere (R03)	**2**
Pais `nvarchar(30)`	derivado da zona	`Espanha` → Espanha; `Norte`, `Centro`, `Sul`, `Ilhas` → Portugal; zona desconhecida → `Desconhecido`	**2** (com a zona)
Tipo_Cliente `nvarchar(20)`	`customers.type`	`limpa`	**2**
Segmento `nvarchar(25)`	`customers.segment`	`limpa`	**2**
Vendedor_Atribuido `nvarchar(30)`	`employees.name` via `customers.seller`	`seller = 0` → `NULL` (4 clientes). O hash SCD2 usa o **número** do vendedor, não o nome	**2**
Dias_Pagamento `smallint`	`customers.days_pay_invoices`	Cópia	1
(excluído)	`customers.BBAN`	Dado financeiro desnecessário (minimização de dados)	—

Duplicados (clientes 10 e 18): são carregados como dois clientes distintos (`stg_customer_duplicate_groups`, estado `PENDING_REVIEW`). Uma futura consolidação reaponta as faturas ao sobrevivente na carga seguinte.

6.2.3 `Dim_Produto` ← `stg_products`, `stg_families`, `stg_categories`

Atributo destino (tipo)	Origem	Transformação / limpeza	SCD
BK_Produto `char(18)`	`products.reference`	Cópia (sem `RTRIM`, para coincidir com `invoices_lines.product`)	—
Designacao `nvarchar(60)`	`products.designation`	`limpa`; `NULL` → `Desconhecida` (2 produtos, sem vendas)	1
BK_Familia `varchar(18)`	`products.family`	`NULL` → `N/D`	—

Atributo destino (tipo)	Origem	Transformação / limpeza	SCD
Familia `nvarchar(60)`	`families.name`	Sem família → `Desconhecida`	**2**
Classificacao_Familia `nvarchar(20)`	`families.classification`	`limpa`; vazio → `NULL` (8 famílias)	**2**
BK_Categoria `int`	`products.category`	`NULL` → `-1`	—
Categoria `nvarchar(35)`	`categories.Category`	Sem categoria → `Desconhecida`	**2**
Classificacao_Produto `nvarchar(20)`	`products.classification`	`limpa`; se vazia usa `families.classification` (*fallback* da decisão 6)	**2**
Taxa_IVA `decimal(5,2)`	`products.vat_rate` (`numeric(2,0)`)	Cópia; `NULL` mantém-se (2 produtos)	**2**
(excluídos)	`stock`, `minimum_stock`, `current_sell_price`, `last_cost_price`	Semi-aditivos ou variáveis; o preço real está no facto	—

6.2.4 `Dim_Fornecedor` ← `stg_supplliers, stg_zones`

Atributo destino (tipo)	Origem	Transformação / limpeza	SCD
BK_Fornecedor `numeric(10,0)`	`supplliers.number`	Cópia	—
Nome `nvarchar(55)`	`supplliers.name`	`limpa`	1
NIF `varchar(20)`	`supplliers.taxpayer_number`	Como no cliente (`N/D` se vazio ou zeros)	1
Tipo_Fornecedor `nvarchar(20)`	`supplliers.type`	`limpa`	**2**
Nacionalidade `nvarchar(20)`	`supplliers.nationality`	`limpa`; vazio → `NULL`	1
Cidade `nvarchar(43)`	`supplliers.city`	`limpa`; vazio → `Desconhecida`	**2**
Zona `nvarchar(50)`	`zones.zone` via `supplliers.zone`	`NULL` → `Zona desconhecida`	**2**
Morada, Codigo_Postal, Contacto, Cargo_Contacto, Email, Telefone, Telemovel	`address`, `zip_code`, `contact`, `contact_position`, `email`, `phone`, `mobile`	`limpa`; vazio → `NULL`	1
Dias_Pagamento `smallint`	`supplliers.days_pay_invoices`	Cópia	1

Membro técnico **SK −1 "Sem fornecedor"** para os 12 produtos com `products.supplyer` nulo.

6.2.5 `Dim_Funcionario` ← `stg_employees`

Atributo destino (tipo)	Origem	Transformação / limpeza	SCD
BK_Funcionario `numeric(6,0)`	`employees.number`	Cópia	—
Codigo `varchar(20)`	`employees.code`	`limpa`	1
Nome `nvarchar(30)`	`employees.name`	`limpa`	1
Email `varchar(100)`	`employees.email`	`limpa`	1
Grupo `nvarchar(20)`	`employees.[group]`	`limpa` (nome reservado: escrever entre parênteses retos)	**2**

Atributo destino (tipo)	Origem	Transformação / limpeza	SCD
Departamento `nvarchar(20)`	`employees.department` (grafia da origem)	`limpa`	**2**
BK_Chefe `numeric(6,0)`	`employees.chief`	`NULL` para a direção (2 funcionários)	**2**
Nome_Chefe `nvarchar(30)`	`employees.name` via `chief`	`NULL` se não há chefe. O hash SCD2 usa o **número** do chefe; se o chefe mudar de nome, o nome é atualizado em todas as versões dos subordinados	**2** (por `BK_Chefe`)

6.2.6 `Dim_Data`, `Dim_Moeda`, `Dim_Taxa_Cambio`

Não vêm da Staging. `Dim_Data` e `Dim_Taxa_Cambio` (EUR = 1) são populadas por 05_carga_dim_data_moeda.sql. `Dim_Moeda`: `invoices.currency = 'EURO'` → `EUR`; qualquer outro valor sem correspondência é rejeitado. Cotações GBP: carga futura.

6.2.7 `Fact_Vendas` ← `stg_invoices` + `stg_invoices_lines`

Atributo destino (tipo)	Origem	Transformação / limpeza
SK_Data_Fatura `int`	`invoices.invoice_date`	`yyyymmdd`; data fora de `Dim_Data` → rejeitar
SK_Data_Pagamento `int`	`invoices.payment_date`	`yyyymmdd`; `NULL` → `-1`. Na amostra a data está sempre preenchida e pode ser de vencimento (a confirmar)
SK_Cliente `int`	`invoices.customer`	Pesquisa em `Dim_Cliente` por `BK_Cliente` e `invoice_date BETWEEN Data_Inicio AND Data_Fim`
SK_Produto `int`	`invoices_lines.product`	Idem, em `Dim_Produto`
SK_Fornecedor `int`	`products.supplyer` do produto da linha	Idem, em `Dim_Fornecedor`; `NULL` → `-1`
SK_Funcionario `int`	`invoices.employee`	Idem, em `Dim_Funcionario`
SK_Moeda `smallint`	`invoices.currency`	`EURO` → `EUR` → `Dim_Moeda`
Numero_Fatura `char(25)`	`invoices.invoice_stamp_id`	Cópia
BK_Linha_Fatura `char(25)`	`invoices_lines.invoice_line_stamp_id`	Cópia; chave do *upsert*
Quantidade `decimal(14,4)`	`invoices_lines.quantity`	Cópia, sem alterar o sinal
Preco_Unitario `decimal(19,6)`	`invoices_lines.unit_price`	Cópia
Valor_Liquido `decimal(19,6)`	`invoices_lines.line_total`	Cópia; R06 (`≠ quantidade × preço`) rejeita a fatura
Valor_IVA `decimal(19,6)`	`invoices_lines.line_vat`	Cópia; `NULL` (permitido na origem, 0 casos) → rejeitar, por ser medida
Frete_Alocado `decimal(19,6)`	`invoices.shipping_cost`	`ROUND(shipping_cost × line_total / Σ line_total, 6)`; resíduo na última linha por `invoice_line_stamp_id`; se `Σ line_total = 0`, divisão igual pelas linhas. Controlo: `Σ = shipping_cost` por fatura
Taxa_Cambio `decimal(18,8)`	`Dim_Taxa_Cambio`	Taxa da moeda no dia da fatura; sem taxa → rejeitar (a FK composta também o impede)

Atributo destino (tipo)	Origem	Transformação / limpeza
Valor_Liquido_EUR, Valor_IVA_EUR, Frete_Alocado_EUR	cálculo	`ROUND(valor × Taxa_Cambio, 6)`
Data_Criacao_Origem	`invoices_lines.create_*`	`datetime2(0)`
Data_Atualizacao_Origem	`invoices_lines.last_update_*` e `invoices.last_update_*`	**O máximo dos dois**: uma alteração ao frete no cabeçalho muda a alocação de todas as linhas sem tocar nas linhas
ID_Lote_ETL `bigint`	`stg_load_control.load_id`	Lote que inseriu ou atualizou
Hash_Linha `varbinary(32)`	`raw_row_hash` da linha e da fatura	`SHA2_256` da concatenação dos dois hashes: a linha só é reaplicada se mudar a linha **ou** o cabeçalho

Limitação conhecida: `Dim_Fornecedor` não faz parte da hierarquia SCD de `Dim_Produto`. O fornecedor da linha é o que `products.supplyer` tem na altura da carga e não se historiciza se o produto mudar de fornecedor.

6.2.8 Algoritmo de carga das dimensões (por lote)

1. Ler da Staging os registos `VALID/CORRIGIDO` do lote com `last_update` posterior à última carga bem-sucedida (ou todos, se for a 1.ª carga).
2. Calcular `Hash_SCD1` e `Hash_SCD2` sobre os valores transformados.
3. Procurar a versão corrente por BK (`Versao_Atual = 1`):
   4. **não existe** → inserir versão com `Data_Inicio = 1900-01-01`;
   5. `Hash_SCD2` **igual e** `Hash_SCD1` **diferente** → atualizar (tipo 1) **todas** as versões desse BK;
   6. `Hash_SCD2` **diferente** → fechar a versão corrente e inserir nova versão (tipo 2) com os atributos SCD1 atuais;
   7. **hashes iguais** → nada a fazer (repetir o lote não altera nada).
8. As dimensões carregam **antes** dos factos.

6.2.9 Colunas que a Staging tem de fornecer (verificação cruzada)

As tabelas tipadas `stg_<origem>` ainda não existem em 03_ddl_staging_area.sql. Para este mapeamento funcionar, têm de existir:

Necessidade do Data Mart	Onde
Todas as colunas de negócio das 9 tabelas acima, com o tipo da origem (incluindo `BBAN`, que **não** sobe para o Data Mart)	`stg_customers`, `stg_products`, `stg_families`, `stg_categories`, `stg_suplliers`, `stg_zones`, `stg_employees`, `stg_invoices`, `stg_invoices_lines`
Colunas técnicas `load_id`, `validation_status`, `validation_rule`, `raw_row_hash`	todas
`frete_alocado`, `line_sum` (Σ das linhas) e `header_line_difference`	`stg_invoices_lines` / `stg_invoices`
Marca de que a classificação do produto veio do *fallback* da família	`stg_products` (decisão 6 pede para registar a origem do valor; o Data Mart não tem coluna para isso)
Estado `PENDING_REVIEW` do grupo de duplicados por cliente	`stg_customer_duplicate_groups` (já existe)

7 Regras de transformação, limpeza e rejeição

ID	Problema observado	Regra proposta	Tratamento previsto
R01	Clientes 10 e 18 com número de contribuinte e nome normalizados iguais	Comparar o contribuinte normalizado e outros atributos de identificação, preservando os identificadores originais.	Classificar como potenciais duplicados. Não consolidar automaticamente nem eliminar qualquer cliente antes de definir uma regra determinística e preservar as referências às faturas.
R02	Email e telemóvel em falta em clientes	Distinguir atributos opcionais de atributos obrigatórios para cada análise.	Preservar os valores nulos. A ausência de contacto, por si só, não deve excluir uma venda do Data Mart, salvo se um requisito analítico justificado o exigir.
R03	Zona em falta em clientes	Verificar o valor de origem e a existência de uma correspondência válida.	Não inferir uma zona. Preservar o valor em falta e definir, no modelo dimensional, o tratamento adequado para a zona desconhecida, se aplicável.
R04	Atributos de produto em falta	Validar designação, família, categoria, fornecedor, stock e preço de venda de acordo com a utilização analítica de cada atributo.	Preservar os valores originais e sinalizar os campos em falta. Não inventar valores comerciais nem rejeitar automaticamente todos os factos associados sem avaliar o impacto no modelo dimensional.
R05	12 linhas de compra sem referência de produto	Identificar referências nulas ou vazias em `purchases_lines.product`.	Manter os registos na Staging Area, encaminhá-los para uma estrutura própria de registos com problemas de qualidade e excluí-los do Data Mart enquanto não for possível identificar o produto de forma fiável. Não atribuir um produto arbitrário.
R06	Quatro linhas de fatura em que quantidade × preço unitário difere de `line_total`	Comparar o valor registado com o cálculo, preservando os sinais e os valores originais.	Sinalizar os quatro casos para investigação. Não aplicar valores absolutos, inverter sinais ou substituir totais sem confirmação da regra de negócio. Se o problema não puder ser resolvido automaticamente e afetar a fiabilidade das medidas, manter os registos na estrutura própria de rejeições da Staging Area e excluí-los do carregamento analítico.
R07	Duas faturas em que o total de produtos do cabeçalho difere da soma das linhas	Calcular e registar a diferença entre o cabeçalho e as linhas, com uma tolerância documentada de 0,01 para comparação monetária.	Sinalizar as duas faturas. Não escolher automaticamente um dos totais. Se a discrepância não puder ser corrigida de forma fiável e comprometer as medidas carregadas no Data Mart, manter os registos afetados na estrutura própria de rejeições da Staging Area e excluí-los do carregamento analítico.
R08	Possíveis referências a entidades inexistentes	Validar as referências não nulas contra as entidades de origem correspondentes.	Os testes atuais não encontraram referências inválidas nas relações verificadas. Se forem detetadas em cargas futuras, sinalizar e manter os registos problemáticos na estrutura própria da Staging Area, excluindo-os do Data Mart até resolução.
R09	Diferenças de representação textual	Aplicar normalização controlada, como remoção de espaços exteriores e normalização de maiúsculas/minúsculas para comparação.	Utilizar valores normalizados para validação e comparação, sem perder os valores originais. Não modificar nomes ou identificadores de negócio de forma irreversível.
R10	Repetição de registos em cargas sucessivas	Usar as chaves primárias da origem e os atributos de criação/última atualização disponíveis para identificar registos novos ou alterados.	Implementar carregamento incremental idempotente. Uma nova execução não deve duplicar registos já carregados; a estratégia concreta depende da validação dos campos de auditoria e será especificada na etapa de carregamento incremental.

Os registos são classificados como correção automática segura, atributo opcional em falta, problema por investigar ou registo rejeitado. Um atributo opcional em falta não rejeita a venda; uma chave, medida ou relação inválida rejeita-a.

7.1 Contrato de carregamento e decisões fechadas

As regras desta especificação são aplicadas pela ordem seguinte:

1. Extrair o valor original e os metadados técnicos para a Staging Area.
2. Normalizar apenas cópias de comparação e calcular os indicadores de validação.
3. Aplicar as correções determinísticas permitidas.
4. Classificar cada registo como `VALID`, `CORRIGIDO`, `PENDENTE` ou `REJEITADO`.
5. Carregar para o Data Mart apenas registos `VALID` ou `CORRIGIDO`.

Ficam fechadas as seguintes decisões para a Fase 1:

- **Clientes 10 e 18:** são potenciais duplicados, mas não existe confirmação de negócio. Não são consolidados nem eliminados automaticamente. Ambos permanecem na Staging e são carregados como clientes distintos, com `duplicate_group_id` comum e estado `PENDING_REVIEW`. Uma futura consolidação exige um mapa explícito entre as chaves de origem e o sobrevivente; até lá, as duas faturas mantêm o respetivo cliente de origem.
- **Quantidades e preços negativos:** não são convertidos para valores absolutos nem reinterpretados como devoluções. As quatro linhas financeiras e as respetivas faturas são `REJEITADO` até existir confirmação da semântica.
- **Divergências de cabeçalho:** as duas faturas com diferença entre o total de produtos e a soma das linhas são rejeitadas integralmente. A rejeição ao nível da fatura preserva a reconciliação do frete alocado.
- **Produto desconhecido em compras:** as 12 linhas ficam em `stg_rejections` com os valores originais; não entram no Data Mart de vendas.
- **Membro desconhecido:** só é usado para atributos descritivos ausentes que não impedem a identificação do facto, por exemplo fornecedor, zona ou contacto. Nunca substitui uma referência de produto, cliente, documento, quantidade, preço ou total inválida.
- **Contactos opcionais:** permanecem `NULL`. Valores vazios são normalizados para `NULL` numa coluna transformada, conservando o valor bruto.
- **Vendas com IVA:** `Valor_Liquido + Valor_IVA`, sem frete. O frete apenas entra em `Total_Faturado` e nas análises que o pedem explicitamente. A origem indica IVA ao nível dos produtos, pelo que não se presume IVA sobre o frete.
- **Frete:** é alocado proporcionalmente ao valor líquido das linhas válidas da fatura, arredondado a 6 casas, e o resíduo é atribuído à última linha por ordem estável de `invoice_line_stamp_id`. Se alguma linha da fatura for rejeitada, a fatura inteira é rejeitada.
- **Moeda:** a taxa é obtida de uma fonte de câmbio versionada por moeda e data da fatura. Para EUR a taxa é 1. Se não existir taxa aplicável, o registo fica rejeitado; não se usa a taxa do dia anterior sem regra documentada.
- **Idempotência:** `source_table + source_pk + load_id` identifica a extração e `source_table + source_pk` identifica o registo lógico. O Data Mart usa as chaves de negócio e `MERGE`/upsert controlado; repetir o mesmo lote não cria factos adicionais.

Estas decisões substituem as formulações de "decisão pendente" existentes nos documentos e devem ser usadas como referência pelos mapeamentos e pelo ETL.

8 Carga incremental

O enunciado exige que cada execução carregue só os dados novos e atualize os que mudaram.

- **Deteção de registos novos ou alterados:** as tabelas de negócio (exceto `zones` e `categories`, que não têm colunas de auditoria e são minúsculas) têm `create_*` e `last_update_*`. Cada extração lê os registos com `last_update` posterior à última carga bem-sucedida registada em `stg_load_control`. `zones` e `categories` são lidas por inteiro e comparadas por *hash*.
- **Idempotência:** `source_table + source_pk` identifica o registo lógico e `raw_row_hash` o seu conteúdo. Repetir um lote não cria nem altera nada.
- **Dimensões:** a comparação do `Hash_SCD1` e do `Hash_SCD2` com a versão corrente decide entre ignorar, atualizar (tipo 1) ou abrir uma nova versão (tipo 2).
- **Facto:** *upsert* por `BK_Linha_Fatura`. O `Hash_Linha` combina os *hashes* da linha e do cabeçalho da fatura, porque uma alteração ao custo de expedição no cabeçalho muda a alocação de todas as linhas sem alterar as linhas.
- **Limitação:** a origem não marca eliminações, pelo que os registos apagados na origem não são detetados.

9 Uniformização monetária em euros

Para suportar a futura expansão para o Reino Unido, os valores monetários do facto são guardados na moeda original e em euros, juntamente com a taxa de câmbio aplicada. A taxa é a da moeda no dia da fatura (`Dim_Taxa_Cambio`, uma linha por moeda e dia); para euros é 1. Na amostra só existe a moeda `EURO` (mapeada para `EUR`); a estrutura e a moeda `GBP` já estão previstas. Sem taxa aplicável, o registo é rejeitado em vez de se usar a taxa do dia anterior. O facto tem uma chave estrangeira composta para as taxas, pelo que a base de dados impede essa situação.

10 Decisões em aberto e limitações

Ponto	Situação
Sinal das 4 linhas de fatura com quantidade ou preço negativo	Rejeitadas até o negócio confirmar se são devoluções
Significado de `payment_date`	Sempre preenchida; pode ser data de vencimento
Tabelas tipadas `stg_<origem>`	Contrato definido; a criação física é feita na 2.ª parte
Marca de classificação de produto vinda do *fallback* da família	A Staging deve registar a origem; o Data Mart não tem coluna para isso
Fornecedor da linha	Reflete `products.supplyer` à data da carga; não é historiado
Estações do ano	Usam datas fixas (20/03, 21/06, 23/09, 21/12)
Eliminações na origem	Não são detetadas

Anexo A — Matriz de requisitos analíticos

Medidas base (todas ao nível da linha de fatura)

Medida	Definição	Aditiva?	Origem
Quantidade	Unidades vendidas	Sim	`invoices_lines.quantity`
Valor líquido	Venda sem IVA e sem frete	Sim	`invoices_lines.line_total`
Valor IVA	IVA da linha	Sim	`invoices_lines.line_vat`
Frete alocado	Custo de expedição da fatura repartido pelas linhas	Sim	`invoices.shipping_cost` (alocado, ver §4)
Preço unitário	Preço de venda da linha	Não (só média)	`invoices_lines.unit_price`

Medidas derivadas (calculadas, não guardadas):

Medida derivada	Fórmula
Vendas com IVA (produtos)	Valor líquido + Valor IVA; não inclui frete
Vendas sem IVA com frete	Valor líquido + Frete alocado
Total faturado	Valor líquido + Frete alocado + Valor IVA (reconcilia com `invoices.final_total`)
Nº de faturas	`DISTINCT COUNT` de `numero_fatura`

> Decisão da Fase 1: "com IVA" é `líquido + IVA` e não inclui frete. O frete só entra quando é pedido explicitamente; na BD, o IVA incide só sobre os produtos.

Matriz das 10 análises do enunciado

Legenda: **DD** = drill down · **RU** = roll up · Todas as análises incluem a dimensão **Data**.

#	Medidas	Filtros	Agrupar por	Hierarquias / operações	Dimensões
1	Vendas com IVA	Semestre = 1, Ano = 2025	Zona (cliente), Categoria	Data: Ano > Semestre · Geografia cliente: Zona · Produto: Categoria	Data, Cliente, Produto
2	Frete alocado	Mês = fev/2025	Cidade (cliente), Família	Geografia cliente: Cidade · Produto: Família	Data, Cliente, Produto
3	Quantidade	Trimestre = 1, Ano = 2025	Fornecedor, Funcionário	— (nível folha)	Data, Fornecedor, Funcionário
4	Valor líquido, Valor IVA	Ano = 2025	Cliente, Funcionário	— (nível folha)	Data, Cliente, Funcionário
5	Vendas com IVA	**Último dia do mês**, Ano = 2025	Categoria	**DD** Categoria → Produto	Data, Produto
6	Vendas com IVA	**Semana 6**, Ano = 2025; Zona cliente = Norte	Família	**DD** Família → Produto	Data, Cliente, Produto
7	Quantidade	**Quadrimestre = 1**, Ano = 2025	Cliente, Produto	**RU** Cliente → Tipo de cliente (e só até aí)	Data, Cliente, Produto

#	Medidas	Filtros	Agrupar por	Hierarquias / operações	Dimensões
8	Valor IVA	**Estação = Primavera**, Ano = 2025	Cidade (cliente), Família	Geografia cliente: Cidade · Produto: Família	Data, Cliente, Produto
9	Valor líquido (sem IVA, sem frete)	Ano = 2025; Categoria ∈ {Ar, Cordas, Membranas}	Cliente e Mês	**RU** Cliente → Zona · **RU** Mês → Trimestre	Data, Cliente, Produto
10	Vendas com IVA, Quantidade	**Fim de semana**, Trimestre = 2, Ano = 2025; Família ∈ {Acessórios, Amplificadores, Teclados}	Zona/Fornecedor e Zona/Cliente	**DD** Zona fornecedor → Fornecedor · **DD** Zona cliente → Cliente	Data, Fornecedor, Cliente, Produto

Atributos de dimensão exigidos pelas análises

Atributo	Dimensão	Análises
Semestre, Trimestre, Quadrimestre, Mês, Semana do ano, Dia, Estação, Ano	Data	1–10
Indicador "último dia do mês"	Data	5
Indicador "fim de semana"	Data	10
Zona, Cidade, Cliente, Tipo de cliente	Cliente	1, 2, 4, 6–10
Categoria, Família, Produto	Produto	1, 2, 5, 6, 7, 8, 9, 10
Fornecedor, Zona do fornecedor	Fornecedor	3, 10
Funcionário	Funcionário	3, 4

Análises adicionais suportadas

O modelo não foi desenhado só para as 10 análises acima. Também responde a:

Análise	Como o modelo a suporta
Evolução mensal/anual e comparação com o período homólogo	Dim_Data com hierarquias completas
Ranking de produtos, famílias e clientes	Dim_Produto, Dim_Cliente + medidas aditivas
Vendas por segmento e por tipo de cliente	`Segmento` e `Tipo_Cliente` em Dim_Cliente
Desempenho por funcionário, grupo e departamento, ou por chefe	Dim_Funcionario com hierarquia de chefia
Ticket médio e nº de produtos por fatura	`numero_fatura` (dimensão degenerada)
Peso do frete nas vendas, por zona, família ou fornecedor	Frete alocado + dimensões
Prazo de pagamento (data pagamento − data fatura)	Duas relações a Dim_Data (papéis)
Devoluções (quantidade negativa)	Medidas aditivas aceitam sinal
Vendas em Espanha vs. Portugal	`Pais` na hierarquia geográfica
Vendas em várias moedas (futuro UK)	Valores em moeda original + EUR, `Dim_Moeda`
Vendas por cliente na data da venda (histórico)	SCD tipo 2 em Dim_Cliente

Nota sobre a amostra

As 37 faturas da amostra têm `invoice_date` entre **31/01/2025 e 01/05/2025**. Algumas análises (por exemplo a 8, primavera, e a 10, 2.º trimestre) vão devolver poucos dados. Isso não é erro do modelo.

Anexo B — Dicionário de dados e estratégias SCD

Dicionário de dados

Convenções: `Hash_SCD1` / `Hash_SCD2` (`varbinary(32)`, não nulos, nas quatro dimensões com SCD) guardam o SHA2-256 das colunas SCD1 e SCD2 normalizadas, para detetar alterações na carga sem comparar coluna a coluna; `SK_` = chave substituta (surrogate key, `int IDENTITY`); `BK_` = chave de negócio vinda da origem. Cada dimensão tem um membro **"Desconhecido"** com SK = −1. **PK** = chave primária, **FK** = chave estrangeira, **N** = nulável. Os tipos são de SQL Server.

`Fact_Vendas`

Granularidade: uma linha de fatura. **Chave natural:** `BK_Linha_Fatura` (única).

Coluna	Tipo	PK/FK	Nulável	Descrição	Origem
SK_Venda	bigint IDENTITY	PK	N	Chave substituta da linha de factos	—
SK_Data_Fatura	int	FK → Dim_Data	N	Data da fatura (papel principal)	`invoices.invoice_date`
SK_Data_Pagamento	int	FK → Dim_Data	N	Data de pagamento (papel secundário). Na origem `payment_date` é sempre preenchida (03/02 a 31/12/2025, nunca anterior à fatura): pode ser data de vencimento, a confirmar com o negócio	`invoices.payment_date`
SK_Cliente	int	FK → Dim_Cliente	N	Versão do cliente à data da venda	`invoices.customer`
SK_Produto	int	FK → Dim_Produto	N	Versão do produto à data da venda	`invoices_lines.product`
SK_Fornecedor	int	FK → Dim_Fornecedor	N	Fornecedor do produto vendido; **−1 ("Sem fornecedor")** quando `products.supplyer` é nulo (12 produtos na amostra)	`products.supplyer`
SK_Funcionario	int	FK → Dim_Funcionario	N	Funcionário que emitiu a fatura	`invoices.employee`
SK_Moeda	smallint	FK → Dim_Moeda	N	Moeda original da fatura	`invoices.currency`
Numero_Fatura	char(25)	— (dim. degenerada)	N	Identificador da fatura	`invoices.invoice_stamp_id`
BK_Linha_Fatura	char(25)	UK	N	Identificador da linha (lineage e carga incremental)	`invoices_lines.invoice_line_stamp_id`
Quantidade	decimal(14,4)	—	N	Unidades vendidas (negativa em devoluções)	`invoices_lines.quantity`

Coluna	Tipo	PK/FK	Nulável	Descrição	Origem
Preco_Unitario	decimal(19,6)	—	N	Preço unitário, moeda original (não aditivo)	`invoices_lines.unit_price`
Valor_Liquido	decimal(19,6)	—	N	Venda sem IVA e sem frete, moeda original	`invoices_lines.line_total`
Valor_IVA	decimal(19,6)	—	N	IVA da linha, moeda original	`invoices_lines.line_vat`
Frete_Alocado	decimal(19,6)	—	N	Frete repartido pela linha, moeda original	calculado (ver §1)
Taxa_Cambio	decimal(18,8)	—	N	Taxa moeda original → EUR no dia da fatura	`Dim_Taxa_Cambio`
Valor_Liquido_EUR	decimal(19,6)	—	N	`Valor_Liquido × Taxa_Cambio`	calculado
Valor_IVA_EUR	decimal(19,6)	—	N	`Valor_IVA × Taxa_Cambio`	calculado
Frete_Alocado_EUR	decimal(19,6)	—	N	`Frete_Alocado × Taxa_Cambio`	calculado
Data_Criacao_Origem	datetime2(0)	—	N	`create_date + create_time` da linha	`invoices_lines.create_*`
Data_Atualizacao_Origem	datetime2(0)	—	N	`last_update_date + last_update_time` (incremental)	`invoices_lines.last_update_*`
ID_Lote_ETL	bigint	—	N	Execução do ETL que inseriu/atualizou (`stg_load_control.load_id`)	ETL
Hash_Linha	varbinary(32)	—	N	`raw_row_hash` da linha na Staging; a mesma linha com o mesmo hash não é reaplicada (idempotência)	Staging

`SK_Data_Pagamento` usa SK −1 ("sem data") quando a fatura ainda não foi paga.

Restrições físicas adicionais (ver 04_ddl_data_mart.sql): FK composta `(SK_Data_Fatura, SK_Moeda) → Dim_Taxa_Cambio`, que impede carregar uma linha sem taxa de câmbio no dia da fatura (regra 7 da Fase 1); `CK_Fact_Taxa_Positiva` (`Taxa_Cambio > 0`); `CK_Fact_Fatura_Dia` (`SK_Data_Fatura <> -1`, a data da fatura é obrigatória).

`Dim_Data` (papéis: Data da Fatura, Data de Pagamento)

Carregada uma só vez (por exemplo 2020–2035). **Sem SCD** (valores imutáveis).

Coluna	Tipo	PK/FK	Nulável	Descrição / exemplo
SK_Data	int	PK	N	`yyyymmdd`, ex.: 20250207
Data	date	UK	N	2025-02-07
Dia_Mes	tinyint	—	N	7

Coluna	Tipo	PK/FK	Nulável	Descrição / exemplo
Dia_Ano	smallint	—	N	38
Dia_Semana	tinyint	—	N	1 = segunda … 7 = domingo (ISO)
Nome_Dia_Semana	varchar(15)	—	N	Sexta-feira
Fim_Semana	bit	—	N	1 se sábado/domingo (análise 10)
Ultimo_Dia_Mes	bit	—	N	1 se último dia do mês (análise 5)
Semana_Ano	tinyint	—	N	Semana ISO 1–53 (análise 6)
Ano_Semana_ISO	smallint	—	N	Ano ISO da semana (difere de `Ano` no início/fim do ano)
Mes	tinyint	—	N	2
Nome_Mes	varchar(15)	—	N	Fevereiro
Ano_Mes	int	—	N	202502 (ordenação)
Trimestre	tinyint	—	N	1–4
Quadrimestre	tinyint	—	N	1–3 (análise 7)
Semestre	tinyint	—	N	1–2 (análise 1)
Estacao_Ano	varchar(10)	—	N	Primavera/Verão/Outono/Inverno (análise 8)
Ano	smallint	—	N	2025
Feriado_Nacional	bit	—	N	1 se feriado em Portugal
Nome_Feriado	varchar(60)	—	**S**	Preenchido só nos feriados

Estações (hemisfério Norte, datas astronómicas): Primavera 20/03–20/06 · Verão 21/06–22/09 · Outono 23/09–20/12 · Inverno 21/12–19/03. Os dias de transição variam de ano para ano e ficam registados na tabela em cada ano.

Membro especial: SK = −1 ("Sem data"), usado em `SK_Data_Pagamento` quando não há pagamento.

`Dim_Cliente`

Origem: `customers` + `zones`. **Um cliente de origem = uma chave de negócio**; potenciais duplicados não são consolidados automaticamente (ver §6).

Coluna	Tipo	PK/FK	Nulável	Descrição	SCD
SK_Cliente	int IDENTITY	PK	N	Chave substituta	—
BK_Cliente	numeric(10,0)	—	N	`customers.number`	—
Nome	nvarchar(55)	—	N	Nome	1
NIF	varchar(20)	—	N	`taxpayer_number` (chave de deteção de duplicados). Em falta/vazio ou `000000000` → `N/D` (1 cliente sem NIF e 1 com `000000000` na amostra)	1
Contacto	nvarchar(30)	—	**S**	Pessoa de contacto	1
Email	varchar(45)	—	**S**	Email	1
Telefone	varchar(60)	—	**S**	`phone`	1
Telemovel	varchar(45)	—	**S**	`mobile`	1

Coluna	Tipo	PK/FK	Nulável	Descrição	SCD
Morada	nvarchar(55)	—	**S**	Morada	1
Codigo_Postal	varchar(45)	—	**S**	Código postal	1
Cidade	nvarchar(43)	—	N	Cidade (`customers.clity`)	**2**
Zona	nvarchar(50)	—	N	Norte, Centro, Sul, Ilhas, Espanha (`zones.zone`). `customers.zone` é nulável na origem (2 clientes na amostra): sem zona → `Zona desconhecida`, nunca inferida	**2**
Pais	nvarchar(30)	—	N	Portugal / Espanha (derivado da zona)	2 (com a zona)
Tipo_Cliente	nvarchar(20)	—	N	Cliente Final, Revendedor, Funcionário, Interno	**2**
Segmento	nvarchar(25)	—	N	Alto, Médio, Baixo	**2**
Vendedor_Atribuido	nvarchar(30)	—	**S**	Nome do funcionário em `customers.seller`; `seller = 0` (4 clientes) significa "sem vendedor" e fica NULL	**2**
Dias_Pagamento	smallint	—	**S**	Prazo de pagamento	1
Data_Inicio	date	—	N	Início de validade da versão	—
Data_Fim	date	—	N	Fim de validade (9999-12-31 se atual)	—
Versao_Atual	bit	—	N	1 se for a versão corrente	—
Data_Criacao_Origem / Data_Atualizacao_Origem	datetime2(0)	—	N	Rastreabilidade	—

> Foi **excluído** o IBAN/`BBAN` (dado pessoal/financeiro desnecessário para análise, segundo a política de minimização de dados).

Hierarquias:

- **Geográfica:** País > Zona > Cidade > Cliente
- **Por tipo:** Tipo de cliente > Cliente (análise 7: *roll up* até ao tipo)
- **Por segmento:** Segmento > Cliente

`Dim_Produto`

Origem: `products` + `families` + `categories`.

Coluna	Tipo	PK/FK	Nulável	Descrição	SCD
SK_Produto	int IDENTITY	PK	N	Chave substituta	—
BK_Produto	char(18)	—	N	`products.reference`	—
Designacao	nvarchar(60)	—	N	Nome do produto	1
BK_Familia	varchar(18)	—	N	`families.reference`	—
Familia	nvarchar(60)	—	N	Nome da família (Amplificadores, Teclados, ...)	**2**

Coluna	Tipo	PK/FK	Nulável	Descrição	SCD
Classificacao_Familia	nvarchar(20)	—	**S**	`families.classification` (Instrumentos, Acessórios, …); vazio nalgumas famílias	**2**
BK_Categoria	int	—	N	`categories.Code`	—
Categoria	nvarchar(35)	—	N	Ar, Cordas, Membranas, …	**2**
Classificacao_Produto	nvarchar(20)	—	**S**	`products.classification`	**2**
Taxa_IVA	decimal(5,2)	—	**S**	`vat_rate` em vigor	**2**
Data_Inicio / Data_Fim / Versao_Atual	date / date / bit	—	N	Controlo SCD2	—
Data_Criacao_Origem / Data_Atualizacao_Origem	datetime2(0)	—	N	Rastreabilidade	—

Excluídos: `stock`, `minimum_stock` (semi-aditivos, pertencem a um modelo de stocks); `current_sell_price` e `last_cost_price` (variam muito, o preço de venda real está no facto).

Hierarquias (duas independentes, porque família e categoria são atributos paralelos do produto):

- **Por família:** Classificação > Família > Produto (análises 2, 6, 8, 10)
- **Por categoria:** Categoria > Produto (análises 1, 5, 9)

`Dim_Fornecedor`

Origem: `suplliers` + `zones`. O fornecedor associa-se à linha **via produto** (`products.supplyer`).

Coluna	Tipo	PK/FK	Nulável	Descrição	SCD
SK_Fornecedor	int IDENTITY	PK	N	Chave substituta	—
BK_Fornecedor	numeric(10,0)	—	N	`suplliers.number`	—
Nome	nvarchar(55)	—	N	Nome	1
NIF	varchar(20)	—	N	`taxpayer_number`	1
Tipo_Fornecedor	nvarchar(20)	—	N	Material / Serviços	**2**
Nacionalidade	nvarchar(20)	—	**S**	Vazia nalguns registos	1
Cidade	nvarchar(43)	—	N	Cidade	**2**
Zona	nvarchar(50)	—	**S**	Zona do fornecedor (análise 10)	**2**
Morada / Codigo_Postal	nvarchar(55) / varchar(45)	—	**S**	Endereço	1
Contacto / Cargo_Contacto	nvarchar(30) / nvarchar(15)	—	**S**	Pessoa de contacto e cargo	1
Email / Telefone / Telemovel	varchar	—	**S**	Contactos	1
Dias_Pagamento	smallint	—	**S**	Prazo de pagamento	1
Data_Inicio / Data_Fim / Versao_Atual	date / date / bit	—	N	Controlo SCD2	—

Hierarquia geográfica: Zona > Cidade > Fornecedor. A Zona é nulável na origem: nesses casos usa-se "Zona desconhecida".

`Dim_Funcionario`

Origem: `employees`.

Coluna	Tipo	PK/FK	Nulável	Descrição	SCD
SK_Funcionario	int IDENTITY	PK	N	Chave substituta	—
BK_Funcionario	numeric(6,0)	—	N	`employees.number`	—
Codigo	varchar(20)	—	N	`code` (login)	1
Nome	nvarchar(30)	—	N	Nome	1
Email	varchar(100)	—	N	Email	1
Grupo	nvarchar(20)	—	N	Vendedor, Administrativo, Administrador	**2**
Departamento	nvarchar(20)	—	N	Armazenagem, Administrativo, Direcção	**2**
BK_Chefe	numeric(6,0)	—	**S**	Número do chefe (vazio para a direção)	**2**
Nome_Chefe	nvarchar(30)	—	**S**	Nome do chefe; o SCD2 é acionado pelo `BK_Chefe`, e um novo nome do mesmo chefe atualiza todas as versões	**2** (por BK)
Data_Inicio / Data_Fim / Versao_Atual	date / date / bit	—	N	Controlo SCD2	—

Hierarquias: Departamento > Grupo > Funcionário, e **Chefe > Funcionário** (hierarquia irregular, pai-filho, achatada em 2 níveis).

`Dim_Moeda` e `Dim_Taxa_Cambio`

Coluna	Tipo	PK/FK	Nulável	Descrição
Dim_Moeda				
SK_Moeda	smallint	PK	N	Chave substituta
Codigo_ISO	char(3)	UK	N	EUR, GBP
Nome	nvarchar(30)	—	N	Euro, Libra esterlina
Simbolo	nvarchar(3)	—	N	€, £
Dim_Taxa_Cambio				
SK_Data	int	PK/FK → Dim_Data	N	Dia da cotação
SK_Moeda	smallint	PK/FK → Dim_Moeda	N	Moeda origem
Taxa_Para_EUR	decimal(18,8)	—	N	1 unidade de moeda = X EUR

`Dim_Moeda` tem **SCD 1** (só correções). As taxas são imutáveis (uma linha por dia), logo sem SCD. Na origem `currency = 'EURO'` é mapeado para `EUR`.

Hierarquias (resumo)

Dimensão	Hierarquia	Níveis
Data	Calendário	Ano > Semestre > Trimestre > Mês > Dia
Data	Quadrimestre	Ano > Quadrimestre > Mês
Data	Semana	Ano ISO > Semana > Dia
Data	Estação	Ano > Estação > Dia

Dimensão	Hierarquia	Níveis
Produto	Família	Classificação > Família > Produto
Produto	Categoria	Categoria > Produto
Cliente	Geografia	País > Zona > Cidade > Cliente
Cliente	Tipo	Tipo de cliente > Cliente
Cliente	Segmento	Segmento > Cliente
Fornecedor	Geografia	Zona > Cidade > Fornecedor
Funcionário	Organização	Departamento > Grupo > Funcionário
Funcionário	Chefia	Chefe > Funcionário

Estratégias SCD — resumo e justificação

A origem regista data/hora de criação e de cada alteração (`create_*`, `last_update_*`), mas **não guarda histórico dos valores anteriores**. O histórico começa por isso no primeiro carregamento. A deteção de alterações faz-se por **comparação com a versão atual** (`Hash_SCD1` / `Hash_SCD2`) nos registos cuja `last_update` seja mais recente que a última carga.

Tipo	Quando se usa	Efeito
SCD 1 (sobrescreve)	Correções e atributos que não explicam resultados passados: nome, NIF, email, telefone, morada, contactos, prazos de pagamento, designação do produto	Atualiza a versão atual; sem histórico
SCD 2 (nova versão)	Atributos que **classificam** o facto e cuja mudança alteraria relatórios passados: zona, cidade, tipo, segmento, vendedor (cliente); família, categoria, classificação, taxa de IVA (produto); zona, cidade, tipo (fornecedor); grupo, departamento, chefe (funcionário)	Fecha a versão atual (`Data_Fim`, `Versao_Atual = 0`) e cria outra
Sem SCD	`Dim_Data`, `Dim_Taxa_Cambio`	Imutáveis

Convenção de vigência (garantida por restrições físicas): a 1.ª versão de cada BK começa em `1900-01-01`, para que factos históricos encontrem versão; ao abrir uma nova versão, a anterior fecha com `Data_Fim = Data_Inicio_nova − 1 dia` (intervalos sem sobreposição, pesquisa por `BETWEEN`). `CK_Dim_*_Vigencia` impõe `Data_Fim ≥ Data_Inicio` e `Versao_Atual = 1 ⇔ Data_Fim = 9999-12-31`; o índice único filtrado `IX_Dim_*_Atual` impede duas versões correntes do mesmo BK.

Justificações-chave:

- **Cliente com SCD 2 em zona/tipo/segmento:** se um cliente passar de "Centro" para "Norte", as vendas anteriores têm de continuar na zona onde foram feitas. Com SCD 1 isso falsificava as análises 1, 6, 9 e 10.
- **Produto com SCD 2 em família/categoria:** uma reclassificação não pode mover vendas históricas de categoria (análises 1, 5, 9).
- **Funcionário com SCD 2 em grupo/departamento:** permite avaliar o desempenho do funcionário no cargo que tinha na altura.
- **Preços fora das dimensões:** o preço real de venda está no facto; um `preco_atual` na dimensão geraria versões a cada alteração sem valor analítico.
- **Facto:** cada linha aponta para a **versão da dimensão vigente na data da fatura**. A pesquisa de SK na carga usa `Data_Fatura BETWEEN Data_Inicio AND Data_Fim`.
- **Atrasos:** se um facto chegar com data anterior à da versão atual (alteração retroativa), usa-se a versão cujo intervalo contém a data da fatura.